# Compiler Construction (CS4031)

**Course Instructor(s):**
Dr. Mehreen Alam, Ms. Nirmal Tariq, Ms. Aminah Ali
**Section(s): A,B,C,D,E,F,G**

# Final Exam

**Total Time (Hrs):** 3
**Total Marks:** 165
**Total Questions:** 7

**Date:** Jun 2, 2026

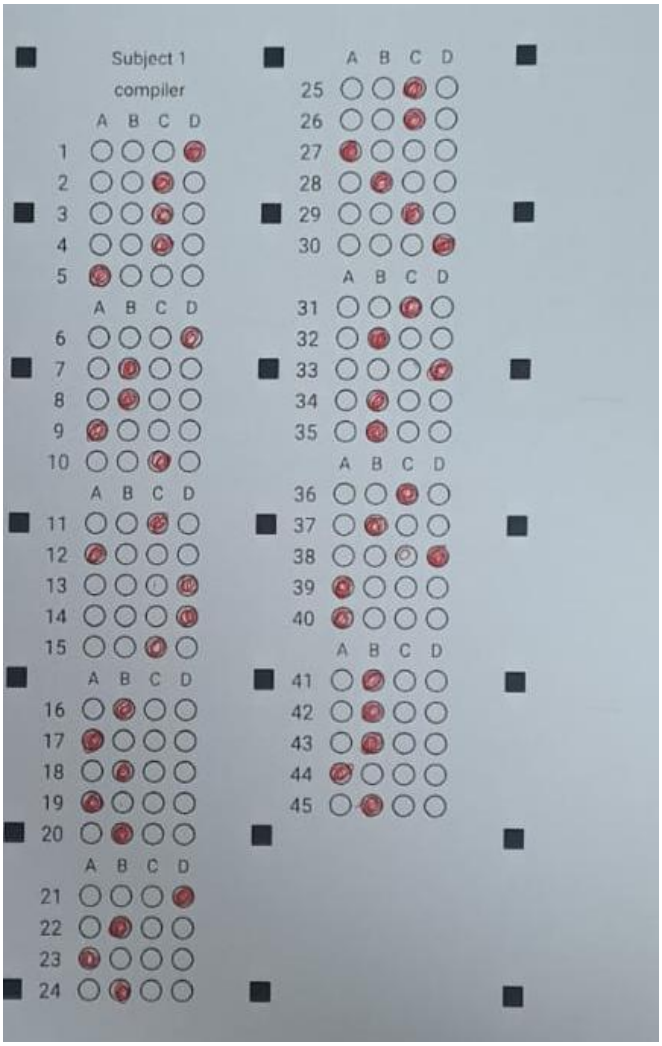
**Roll No** _____ **Course Section** _____ **Student Signature** _____

**Important Information:** **The use or possession of mobile phones and other electronic devices during the exam is strictly prohibited, whether the device is powered on or off. Do not bring mobile phones, smartwatches, tablets, or other electronic gadgets into the examination hall. If you are found with any prohibited devices, your exam may be cancelled, and further disciplinary action may be taken.**
**Do not write below this line.**

---

**SOLUTION:**

**QUESTION 1 MCQS:**



**QUESTION 2:**
**Q1: Compute First and Follow of the Grammar (10 Marks)**
**S→A B c**
**A→a A b | B c**
**B→b B | ε**
**C→c A | S d**
**D→A B | C d | ε**

**Note : If any terminal is missing in set of First of Follow OR if any terminal is written extra the set is considered wrong because it totally changes what strings are accepted while parsing.**

| | First | Follow |
|---|---|---|
| S | {a,b,c} | {$,d} |
| A | {a,b,c} | {b,c,d,$} |
| B | {b,ε} | {c,$} |
| C | {a,b,c} | {d} |
| D | {a,b,c,ε} | { } |

**Q2: Check if following are LL(1) grammars or not (5 marks )**

**1. S → a S b S | ε**
**Not LL(1) Hidden Ambiguity**

---

**2. S → S a | b**
**Not LL(1) - Left recursion**

---

**3. S → i f C t h e n S | i f C t h e n S e l s e S | other**
**Not LL(1) -Left Factoring Missing - Ambiguous**

---

**4. S → if ( E ) S S' | other**
**S' → else S | ε**
**E → 0 | 1**

**Not LL(1) First/Follow conflict on S`**

---

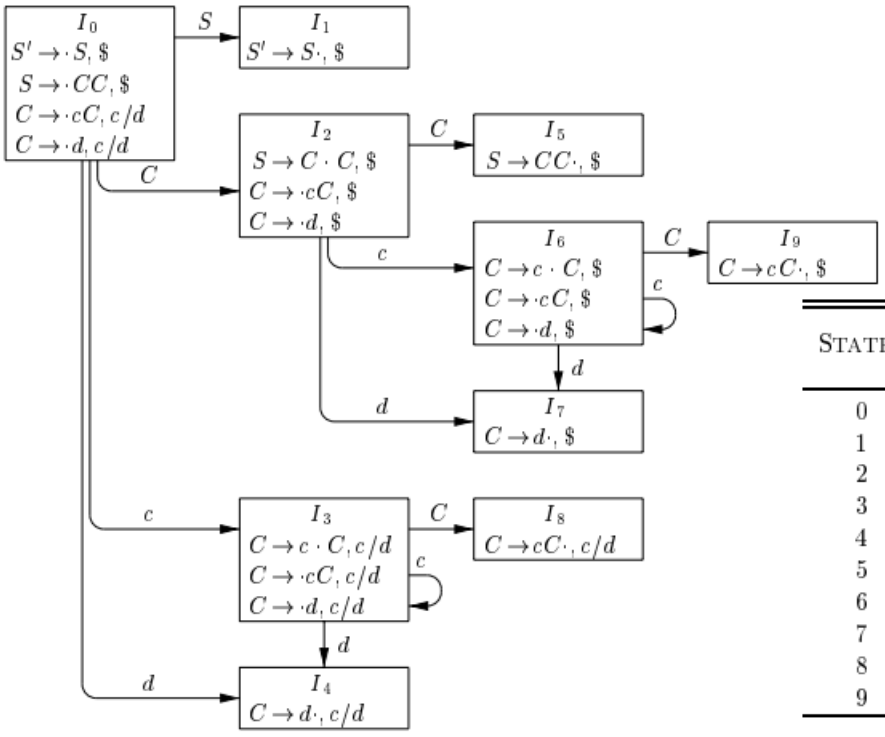
**5. S → A B**
**A → a | ε**
**B → b | ε**
**Yes it is LL(1)**

---

**Q3: Write the corresponding EBNF. (5 marks)**
**list→list , item | item**
**item→item : factor | factor**
**factor→( list ) | id | num**

| |
|---|
| **list = item { “,” item} (2 marks )** |
| **Item = factor {“:” factor } (2 marks )** |
| **factor = “(“ list “)” | “id”| “num” (1 mark)** |

**QUESTION 3:**
**Consider a as c here. Use this as reference.**




| STATE | ACTION | | | GOTO | |
|---|---|---|---|---|---|
| | *c* | *d* | $ | *S* | *C* |
| 0 | s3 | s4 | | 1 | 2 |
| 1 | | | acc | | |
| 2 | s6 | s7 | | | 5 |
| 3 | s3 | s4 | | | 8 |
| 4 | r3 | r3 | | | |
| 5 | | | r1 | | |
| 6 | s6 | s7 | | | 9 |
| 7 | | | r3 | | |
| 8 | r2 | r2 | | | |
| 9 | | | r2 | | |

$$
\begin{array}{lll}
I_{36}: & C \to c \cdot C, & c/d/\$ \\
 & C \to \cdot cC, & c/d/\$ \\
 & C \to \cdot d, & c/d/\$
\end{array}
$$

$I_4$ and $I_7$ are replaced by their union:

$$
I_{47}: \quad C \to d \cdot, \quad c/d/\$
$$

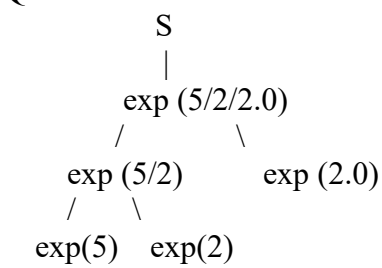
and $I_8$ and $I_9$ are replaced by their union:

$$
I_{89}: \quad C \to cC \cdot, \quad c/d/\$
$$

The LALR action and goto functions for the condensed sets of items are shown in Fig. 4.43.

| STATE | ACTION | | | GOTO | |
|---|---|---|---|---|---|
| | $c$ | $d$ | \$ | $S$ | $C$ |
| 0 | s36 | s47 | | 1 | 2 |
| 1 | | | acc | | |
| 2 | s36 | s47 | | | 5 |
| 36 | s36 | s47 | | | 89 |
| 47 | r3 | r3 | r3 | | |
| 5 | | | r1 | | |
| 89 | r2 | r2 | r2 | | |

QUESTION 4:




**isFloat edges (synthesized — bottom up):**

exp(5).isFloat ⟶ exp(5/2).isFloat
exp(2).isFloat ⟶ exp(5/2).isFloat
exp(5/2).isFloat ⟶ exp(5/2/2.0).isFloat

exp(2.0).isFloat ⟶ exp(5/2/2.0).isFloat
exp(5/2/2.0).isFloat ⟶ S (used to compute exp.etype at S)
**etype edges (inherited — top down):**
exp(5/2/2.0).isFloat ⟶ exp(5/2/2.0).etype (via S rule)
exp(5/2/2.0).etype ⟶ exp(5/2).etype
exp(5/2/2.0).etype ⟶ exp(2.0).etype
exp(5/2).etype ⟶ exp(5).etype
exp(5/2).etype ⟶ exp(2).etype
**val edges (synthesized — bottom up, depends on etype):**
exp(5).etype ⟶ exp(5).val
exp(2).etype ⟶ exp(2).val
exp(5).val ⟶ exp(5/2).val
exp(2).val ⟶ exp(5/2).val
exp(5/2).etype ⟶ exp(5/2).val
exp(5/2).val ⟶ exp(5/2/2.0).val
exp(2.0).val ⟶ exp(5/2/2.0).val
exp(5/2/2.0).etype ⟶ exp(5/2/2.0).val
exp(5/2/2.0).val ⟶ S.val

| Node | isFloat (Pass 1) | etype (Pass 2) | val (Pass 2) |
|---|---|---|---|
| exp → 5 | **false** | **float** | **5.0** |
| exp → 2 | **false** | **float** | **2.0** |
| exp → 2.0 | **true** | **float** | **2.0** |
| exp → 5/2 | **false** | **float** | **2.5** |
| exp → (5/2)/2.0 | **true** | **float** | **1.25** |
| S | — | — | **1.25** |

## QUESTION 5:

### (a) Write the type expressions for variables A and B. [4 Marks]

| Variable | Type Expression |
|---|---|
| A | `array(1..4, array(1..4, int))` |
| B | `array(1..4, array(1..4, int))` |

### (b) Determine the type and validity of each statement. [6 Marks]

| Statement | Type | Valid / Invalid |
|---|---|---|
| i := A[i][i] + 5 | **void** | **Valid** |
| x := A[i][i] | **type_error** | **Invalid** |
| while flag do i := A[i][i] | **void** | **Valid** |

### (c) Determine the type of each expression. [6 Marks]

| Step | Expression | Type |
|---|---|---|
| 1 | A[i] | `array(1..4, int)` |
| 2 | A[i][i] | `int` |
| 3 | A[i][i] + x | `real` |
| 4 | A[i][i] mod x | `type_error` |
| 5 | 5 | `int` |
| 6 | A[i][i] + 5 | `int` |

**(d) Are A and B (i) structurally equivalent and (ii) name equivalent? Justify briefly. [4 Marks]**

| Question | Yes / No | One-line Justification |
|---|---|---|
| Are A and B structurally equivalent? | **Yes** | Expanding type name Row in A gives array(1..4,array(1..4,int)) — identical structure to B |
| Are A and B name equivalent? | **No** | A is declared via type name Matrix->Row; B uses inline array expressions — type names differ |

**QUESTION 6:**

```
for(i=1; i<n; i++)
      for(j=1; j<n; j++)
            c[i,j] = a[i,j] * b[i,j];
print("done");
```

**QUESTION 7:**

**Sol Part1:**

**call by reference means that you transfer pointers to the actual parameters and thus m will point to the space where i + 1 is stored and n will point to where i is stored. The result will be i = 3.**

**call by value means that you copy the actual parameters' values and that odd can not affect the caller's parameters. The result will be i = 3.**

**call by value/result means that you copy the values of the actual parameters to the formal parameters, run the procedure and then copy the value of the formal parameters to the actual parameters. The result will be i = 4.**

**call by name is based on textual substitution; m and n will be exchanged for i + 1 and i before odd is executed. The result will be i = 5.**

**Sol Part 2:**

| | display | stack |
|---|---|---|
| 2 | ? | |
| 1 | • | ole |

| | display | stack |
|---|---|---|
| 2 | • | dole |
| 1 | • | ole |

**At L1:**
**Stack contains the activation record of ole.**
**display[1] points to ole's activation record.**
**display[2] is not yet set for dole.**
**At L2:**
**dole has been called from ole.**
**Stack top contains dole's activation record.**
**Below it is ole's activation record.**
**display[1] points to ole.**
**display[2] points to dole.**
**When dole is called, the old value of display[2] is saved in dole's activation record, and display[2] is updated to point to dole. When dole returns, the old display[2] value is restored.**

**Sol Part 3:**
**Activation tree:**

**fact(4)**
**fact(3)**
**fact(2)**
**fact(1)**
**└── fact(0)**

**At the deepest point, before any return, the stack contains 5 activation records:**

**Top of stack / SP**
**AR fact(0): n = 0, return value = pending, return address → fact(1), FP → fact(0)**
**AR fact(1): n = 1, return value = pending, return address → fact(2), FP → fact(1)**
**AR fact(2): n = 2, return value = pending, return address → fact(3), FP → fact(2)**
**AR fact(3): n = 3, return value = pending, return address → fact(4), FP → fact(3)**
**AR fact(4): n = 4, return value = pending, return address → caller, FP → fact(4)**
**Bottom of stack**

**The current frame pointer points to the activation record of fact(0).**
**The stack pointer is at the top of the stack.**
**Each activation record contains parameter n, return value space, return address, control link/saved FP, and temporary information needed for multiplication.**

**Return values:**
**fact(0) = 1**
**fact(1) = 1 × 1 = 1**
**fact(2) = 2 × 1 = 2**
**fact(3) = 3 × 2 = 6**
**fact(4) = 4 × 6 = 24**

**Final return value of fact(4) = 24.**